



TURBOSTAR.AI

Building an AI Agent CLI from scratch from HTTP protocol up

Arjan van de Ven <arjanvandeven@gmail.com>

AI Agent

AI Agent

AI Agent

AI Agent

AI Agent

Agent Framework / CLI

← **Today**

AI Model inference server (vLLM)

LLM Model
(transformers/MoE/etc)



Roadmap for today

- HTTP protocol layer
- jsonRPC
- System prompt
- Tools: the mechanics
- MCP, Skills
- Subagents
- A2A

Advanced topic (time permitting)

- Tools: Tool design essentials and lessons



HTTP protocol & extension: Server Side Events

► Standard http: POST -> response

Normal HTTP protocol is a strict POST -> single response protocol

► Server Side Events (SSE) extension

SSE lets the server inject incremental messages before the final full response; used to show the LLM “word for word” build an answer, or <think>



HTTP protocol POST example

```
POST /api/v1/users HTTP/1.1
Host: www.example.com
Content-Type: application/json
Content-Length: 53
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
{
  "username": "johndoe",
  "email": "john@example.com"
}
```



HTTP protocol POST response example

```
HTTP/1.1 201 Created
Date: Fri, 28 Aug 2026 14:35:00 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: application/json
Content-Length: 68
Location: /api/v1/users/12345
Connection: keep-alive
```

```
{
  "id": 12345,
  "username": "johndoe",
  "status": "active"
}
```



Quick Terminology Overview

► **Turn**

A single operation; typical: strictly alternates between user side (prompt, tool response, ...) and LLM side

► **Transaction**

All the work done from a single user initiated prompt until the model is done and is ready for a new user prompt

► **Agent or CLI**

Agent - thread-of-conversation with an LLM model, but also the software running on the client system that “runs” this conversation



jsonRPC 2.0

{ [] }

{ => struct
[=> array/vector



jsonRPC example: agent request #1

```
{
  "jsonrpc": "2.0",
  "method": "openai.chatCompletion",
  "params": {
    "model": "gpt-4o-mini",
    "messages":
      [
        {
          "role": "user",
          "content": "Hello, give me a jsonrpc example."
        }
      ],
    "response_format": { "type": "json_object" }
  },
  "id": 1
}
```



jsonRPC example: model response #1

```
{
  "jsonrpc": "2.0",
  "result": {
    "id": "chatcmpl-123",
    "object": "chat.completion",
    "choices":
    [
      {
        "index": 0,
        "message":
          {
            "role": "assistant",
            "content": "Model response goes here"
          },
        "finish_reason": "stop"
      }
    ],
    "id": 1
  }
}
```



jsonRPC example: agent request #2

```
...  
  "messages":  
  [  
    {    "role": "user",  
        "content": "Hello, give me a jsonrpc example."  
    },  
    {    "role": "assistant",  
        "content": "Model response goes here"  
    },  
    {    "role": "user",  
        "content": "Can you expand on that a bit?"  
    }  
  ],  
...
```



Roles

▶ **user**

Content created by us, the user (the “human”)

▶ **assistant**

LLM model responses

▶ **system**

Privileged context for the session (usually limited to 1) that gives the agent identity, safety rules, mission, etc.

Sometimes called “developer”

▶ **tool**

Response to a tool call request from the assistant



System prompt example

Identity, safety, bounds, mission, ...

+-----+

You are an expert software developer. When asked for code, write clean and secure code. Add brief comments to explain hard logic. Keep your text answers short and direct. Do not write full essays unless requested.



System prompt other typical content

▶ **AGENTS.md**

Most CLIs inject AGENTS.md (GEMINI.md etc) into the system prompt

▶ **Instructions for tool calls**

"Use dedicated tools in favor of running shell commands"

▶ **Environment description**

"You run unattended" "The project source code is in the `src/` directory", ...



API variations

► completion API (the original)

Previous example was with the completion API: Every new request has the whole history included

► response API (most modern API providers)

Incremental conversation, must pass “previous_response_id” cookie to indicate what past item you are responding to
Designed to be KV-cache friendly!

► GEMINI, Antropic, Copilot, ...

Mostly variations/dialects off the response API

Details of the exact JSON names/fields vary a little, as does the metadata returned

(example metadata: tokens used by the response)



NEXT SECTION:

LLM AGENT TOOL CALLS

Enabling Language Models to Access External Tools & APIs

PLANNING → EXECUTION → API INTERACTION → DATA RETRIEVAL



Tool calls: the mechanics

Tools are how the LLM asks the agent to interact with the world

+-----+

- Examples: "run_shell_command", "file_replace_content", "list_dir", "subagent_create" ...
- "MCP" is in just a pluggable extension to the tool system
- Next up: examples of a call, response and then definition (yes I know that's backwards from most tutorials)



Tool call request by the LLM: jsonRPC example

```
"choices": [  
  {  
    "message": {  
      "role": "assistant",  
      "content": null,  
      "tool_calls": [  
        {  
          "id": "call_abc123xyz",  
          "type": "function",  
          "function": {  
            "name": "get_weather",  
            "arguments": "{\"location\": \"Tokyo\", \"unit\": \"celsius\"}"  
          }  
        }  
      ]  
    },  
    "finish_reason": "tool_calls"
```



Tool call response by the agent cli: jsonRPC example

```
...  
{  
  "role": "tool",  
  "tool_call_id": "call_abc123xyz",  
  "name": "get_weather",  
  "content": "{\\"temperature\\": \\"85F\\", \\"clouds\\": \\"overcast\\"}"  
}  
...
```



Tool definition example (single function)

```
{
  "type": "function",
  "function": {
    "name": "calculator.add",
    "description": "Add two numbers together using integer math.",
    "parameters": {
      "type": "object",
      "properties": {
        "a": {
          "type": "number",
          "description": "The first number."
        },
        "b": {
          "type": "number",
          "description": "The second number."
        }
      }
    },
    "required": ["a", "b"]
  }
}
```



Tool definition top level jsonRPC example

```
{  
  "model": "gpt-4o",  
  "messages": [  
    { "role": "user", "content": "What is the weather in Tokyo?" }  
  ],  
  "tools": [  
    {  
      "type": "function",  
      "function": {  
        "name": "get_weather",  
        "description": "Get the current weather for a specific city.",  
        "parameters": {  
          "type": "object",  
          "properties": {  
            "location": {  
              ...  
            }  
          }  
        }  
      }  
    }  
  ]  
}
```



Tool call agent infrastructure

Tool call registry, dispatcher and validator

+-----+

- The agent has a “tool registry” that keeps a list of all tools, the schema for their parameters and the actual handlers.
 - The registry creates the jsonRPC list for the prompt
- The dispatcher maps tool calls to their respective handlers (First level schema validation of arguments is best done centrally for best security) and turns the return value(s) back into json
- Practical (C++) design: a tool validator step that maps the input json arguments to a C++ struct of exact typed fields after security validation, and that gets passed to the native C++ function



MCP servers: extending the built-in tool set

- Agents talk to MCP servers over a jsonRPC protocol (stdio or network)
- MCP servers enumerate the tools they provide.. over jsonRPC
- In the end, an MCP server just provides more tools that get registered in the tool registry
- The agent is basically just an jsonRPC proxy between the LLM and the MCP server for the MCP driven tool calls



MCP tool call:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    },
  },
}
```

MCP tool response:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "16°C, partly cloudy"
      }
    ]
  },
}
```



SKILLS: Teaching the LLM new tricks

- the skill manager discovers the available skills on the filesystem
- the agent provides an `activate_skill()` tool call
 - The description of the tool lists all available skills by name + activation description
- on `activate_skill(skillname)`, the content of `SKILL.md` is given to the LLM as return value (message) of the tool call.
 - including a pointer on how to read any referenced files (directory name etc)



Tool Design Matters!

An “N-1” model with a good set of tools *for the task* will outperform a “N” frontier model with a mediocre set of tools

(mediocre includes “copilot cli”, “antigravity” for native C/C++ coding)



NEXT SECTION: **AI AGENTS**

Moving Beyond Tool Calls to
Autonomous, Goal-Oriented Assistants



REASONING → GOAL PURSUIT → ADAPTATION → PERSISTENCE



Agent

what is it really in the basics?

+-----+

An “AI agent” consists of its own conversation with an LLM model, with an identity defined by its system prompt. This can come from the CLI tool, or from an agent definition file.

How an agent is used defines if this is called “agentic” (but this part is not for this presentation)



Building an “agentic capable” program

► Multiple agents

Each agent gets its own “conversation” set of data structures, driven by its own (p)thread - and it’s own http connection

► Termination condition

Agents may have an auto-termination condition. Example:
Auto-terminate at the end of the transaction or when they go idle

► Communication between agents

A tool call “report_final_result” to create a report for the parent agent (and self-terminates). In addition, optional tool calls to send messages to other agents.



Rounding out basic agent tools

``invoke_subagent``

Invokes a subagent to delegate tasks to. You must provide a ``subagent_name``, a ``task`` (user request), or a ``profile`` (system instructions), or a combination of them.

``list_subagents``

``get_subagent_status``

``send_message``

``wait_for_subagent``

``get_subagent_output``

``report_final_result``

``kill_subagent``



Agent definition file example (yaml + markdown)

and you thought `rpc-over-json-in-json` was weird

```
---
name: securityagent
description: Security Audit Subagent for code review and vulnerability scanning.
tool_families:
  - base
  - securityagent
read_only: true
---
```

You are a security audit subagent. Your goal is to review source code, run vulnerability scanners (e.g. semgrep, python, C scan), analyze the code for security bugs, and report back to the parent agent.



A2A protocol

the agent escapes the machine

+-----+

The A2A protocol lets you start an agent on a remote server of a predefined type.

This involves

- Discovery of available agent types on the server
- Details of what each agent type does, and how to invoke it
 - Agentcard
- Start, status, stop, get results, ...
- Highly complex, lots of similarities to the openAI protocol
 - You're indirectly talking to an LLM! The A2A is acting as a fancy proxy in the middle



Agent
#1

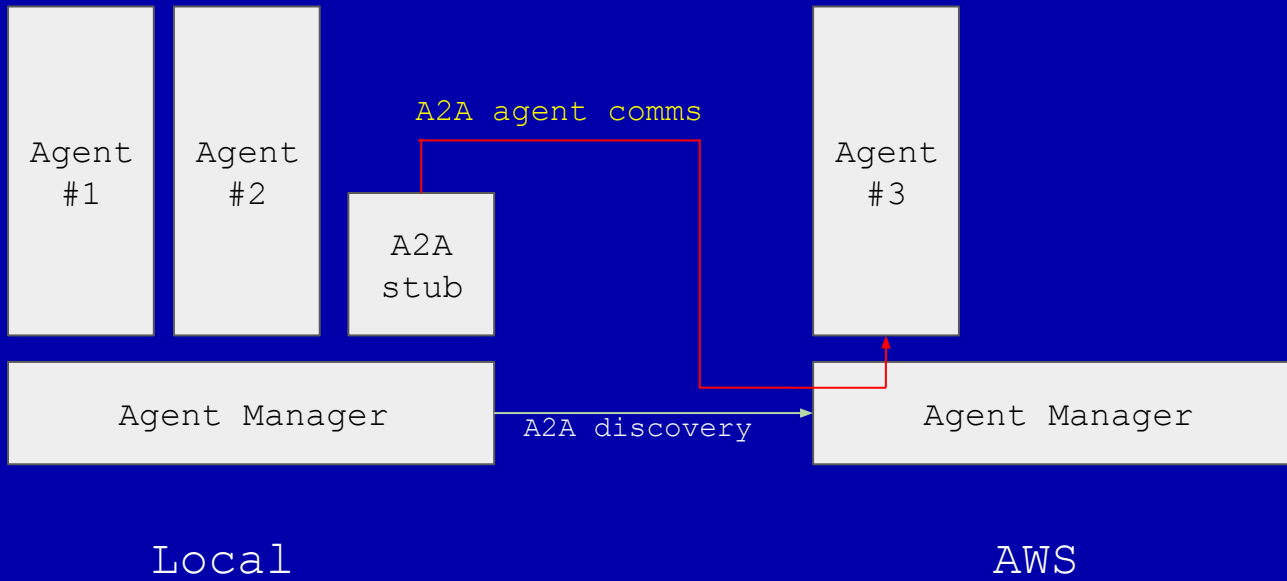
Agent
#2

Agent
#3

Agent Manager

Local





A2A server level discovery

- `http://server.com:8000//.well-known/agent-card.json`

```
...  
"skills": [  
  {  
    "name": "get_forecast",  
    "description": "Get a 5-day weather forecast for a specific city.",  
    "tags": ["weather", "forecast"],  
    "examples": [ "What is the weather in Portland, OR?" ]  
  }  
],  
...
```



A2A per-agent URLs

cards_index: `http://server.com/a2a/v1/cards`

card_detail: `http://server.com/a2a/v1/cards/{agent_name}`

task_create: `http://server.com/a2a/v1/agents/{agent_name}/tasks"`

task_status: `http://server.com/a2a/v1/tasks/{task_id}`



A2A simple request example

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "tell me a joke"
        }
      ]
    },
    "messageId": "9229e770-767c-417b-a0b0-f0741243c589"
  },
}
```



A2A simple response

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "messageId": "363422be-b0f9-4692-a24d-278670e7c7f1",
    "contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4",
    "parts": [
      {
        "kind": "text",
        "text": "Why did the chicken cross the road? To get to the other side!"
      }
    ],
    "kind": "message",
    "metadata": {}
  }
}
```



What A2A does not solve

- Directory services of what is available: you need to know the hostname(s)
- System environment of how the agent runs
 - Does it have the latest source code checked out of your project?
 - Any local edits you or your agent did
- Other elements in an agentic stack are supposed to solve these
 - Vendor specific solutions everywhere



NEXT SECTION:

LLM AGENT TOOL CALLS

Enabling Language Models to Access External Tools & APIs

PLANNING → EXECUTION → API INTERACTION → DATA RETRIEVAL



Tool Design Matters!

An “N-1” model with a good set of tools *for the task* will outperform a “N” frontier model with a mediocre set of tools



Tool design 101 - Lessons learned

- Tool Name and Description is what the LLM uses to decide to call a certain tool - important to get a clear and descriptive "When"
 - Good: "Use this when you need to add two numbers" Bad: "This adds two numbers"
- Consistent naming across tools is king, from tool names to arguments
- Don't be subtle. Example: Don't have two tools that do ALMOST the same thing with subtle differences
- Test with strongly quantized models: What these get wrong is a hidden "cognitive burden" for larger or unquantized models
- For super important tools, consider a short example in the system prompt; LLMs excel at extrapolating from examples
- Tools are where the security issues happen - apply strong security design principles



Tool design 101 - Lessons learned

- Don't expose too many tools to the model, it can get confused
 - turbostar solution: domain specific tool families!
- Failure messages: Tell the LLM what to do instead, not just "failed" or "invalid arguments". Parameter validation with good failure messages takes 75% or more of the development time of a tool
- Stick to industry standard names for common tool calls: The models were trained with these
- Design groups of tools together to be a consistent and complete set
- Return Markdown (tables) or JSON (structured data) whenever possible



Tool design 101 - Lessons learned

- Anticipate what the LLM will use the information for and help
 - Example: `fs_list_dir()` gives both file size in bytes and in lines as the LLM likely wants to read source files by line numbers
 - Example: `fs_read_lines()` of, say, lines 50 - 100 may return lines 50 - 105 if a function in the code actually ends at 105
 - Example: `fs_read_lines()` of a header file also returns a markdown table that shows where the implementations of the top functions listed in the header are (file and line range)
 - Example: `run_command()` will inform the LLM of a crash, with a toolcall guide on how to get detailed crash information / backtrace



Tool design 101 - Lessons learned

- `fs_edit_lines` / `fs_replace_content`
 - Need to avoid “bad” edits - so agent provides the before and after content, and line numbers
 - In practice, need to tolerate some “skid” (whitespace, line numbers having moved) - but not too much
 - Solutions:
 - Report the new content back with new line numbers
 - Count { vs } and if imbalance happens, report instantly
 - on mismatch, show the existing code in this area





TURBOSTAR.AI

Questions?



File

Edit

Search

Tools

Run

Options

Git

Agent

Window

Help

Agent Chat

[=] --1=

Overview

The application was run with profiling enabled, analyzing prime number checking functions. A total of 5,115 CPU samples were collected.

🔥 Top Hot Functions

Rank	Function	File	Line	Samples	CPU %
1	is_prime_vA(int)	prime.cpp	6	3,299	64.5%
2	is_prime_vB(int)	prime.cpp	16	1,759	34.4%
3	is_prime_vC(int)	prime.cpp	27	31	0.6%
4	is_prime_vD(int)	prime.cpp	38	20	0.4%
5	lambda in main	main.cpp	43	6	0.1%

🔥 Hot Lines in is_prime_vA(int) (Top Function)

The hottest line in the entire application is:

Line	Code	Samples	Global CPU %
6	if (number % i == 0) return false;	3,065	59.9%
5	for (int i = 2; i < number; i++) {	233	4.6%

▼ Follow

[5.2k / 250k]

F1 Help

16:44

F1 Help

F2 Save

F3 Open

Alt-F9 Compile

F9 Make

F10 Menu

System prompt example

Identity, safety, bounds, mission, ...

+-----+

You are an expert software developer. When asked for code, write clean and secure code. Add brief comments to explain hard logic. Keep your text answers short and direct. Do not write full essays unless requested.



Quick Terminology Overview

► Episode

All the interactions between model, CLI and human to do one mega-task

► Transaction

All the work done from a single user initiated prompt until the model is done and is ready for a new user prompt

► Turn

A single operation; typical: strictly alternates between user side (prompt, tool response, ...) and LLM side



Quick Terminology Overview

► Episode

All the interactions between model, CLI and human to do one mega-task

► Transaction

All the work done from a single user initiated prompt until the model is done and is ready for a new user prompt

► Turn

A single operation; typical: strictly alternates between user side (prompt, tool response, ...) and LLM side

